

Informe Técnico - Etapa 1

Construcción de la Base de Conocimiento

Equipo: Cristian Cioran & Mateo | Agosto 2026

1. Introducción y Contexto

El presente documento describe las decisiones de diseño adoptadas por el equipo para la construcción de la base de conocimiento vectorial correspondiente a la Etapa 1 del CODEFEST AD ASTRA 2026. El corpus provisto por ADL comprende 1,826 documentos distribuidos en tres fenómenos temáticos: (1) Inteligencia Artificial e Innovación en Entornos Militares, (2) Seguridad Espacial y Órbita Baja Terrestre, y (3) Dinámicas Territoriales en América Latina. Los documentos abarcan formatos PDF, HTML, JSON, CSV, XLSX y TXT en tres idiomas: español, inglés y portugués.

El objetivo principal fue construir un sistema de recuperación híbrido (vectorial + léxico) que maximice las métricas NDCG@10 (nivel fragmento) y F1@3 (nivel documento) sobre las 50 consultas de evaluación, sin utilizar modelos generativos en ninguna etapa del pipeline.

2. Extracción de Texto y Preprocesamiento

La extracción de texto se realiza mediante el módulo `lib/extractor.py`, que implementa un pipeline diferenciado según el formato del archivo:

- PDF: Se utiliza la librería `pypdf` (versión 4+) para extraer texto página por página, preservando el orden de lectura. Se migra desde `PyPDF2` para evitar corrupción de caracteres con tildes y diacríticos en documentos en español y portugués.
- HTML: Se aplica limpieza de etiquetas, scripts y estilos, conservando únicamente el texto visible y los elementos estructurales (encabezados, párrafos).
- JSON: Se interpretan los campos `title`, `body_text` y `body_paragraphs`, concatenándolos en orden de aparición. Los metadatos (`url`, `date`, `authors`) se preservan como metadatos del documento.
- CSV/XLSX: Se lee la cabecera y cada fila se convierte en pares `columna:valor`, tratando cada fila como unidad de fragmentación independiente.
- TXT/Markdown: Se procesa como texto plano, utilizando encabezados como señales de segmentación.

2.1 Limpieza y Normalización

Se aplican los siguientes pasos de normalización:

- Eliminación de caracteres de control y espacios redundantes mediante expresiones regulares.
- Normalización a codificación UTF-8 en todo el pipeline.
- Detección del idioma predominante del documento para asignar stopwords multilingües (español, inglés, portugués) al recuperador léxico BM25.
- Eliminación de elementos repetitivos: cabeceras de página, pies de página, numeración y boilerplate.
- Desduplicación de archivos mediante hash SHA-256 del contenido para detectar y eliminar documentos duplicados que aparecían en múltiples carpetas del corpus.

2.2 Identificación de Documentos

Cada documento recibe un identificador unico (doc_id) con formato F{fenomeno}-{coleccion}-{numero}, por ejemplo F1-AIINDEX-023. Este identificador vincula permanentemente cada archivo con su texto original, fenomeno tematico y fuente de origen.

3. Estrategia de Chunking

La fragmentacion del texto se implementa en lib/chunker.py utilizando una estrategia hibrida por oraciones con control de tamano maximo. Esta estrategia fue seleccionada por las siguientes razones:

- Completitud linguistica: Cada fragmento contiene exclusivamente oraciones completas. Ningun chunk corta una oracion a la mitad, cumpliendo el requisito obligatorio de la Seccion 3.3 de la especificacion tecnica.
- Coherencia semantica: Al respetar los limites oracionales, cada fragmento captura ideas cohesivas que generan embeddings semanticamente precisos.
- Compatibilidad con el encoder: El tamano maximo de tokens por fragmento (512) se alinea con la ventana de contexto del encoder intl-e5-small-v2.

3.1 Proceso de Segmentacion

El segmentador utiliza un regex mejorado para detectar limites oracionales que contempla puntos, signos de interrogacion, signos de exclamacion, y abreviaciones comunes en los tres idiomas. Las oraciones se agrupan secuencialmente hasta alcanzar el limite de 512 tokens, momento en el cual se retrocede al final de la ultima oracion completa. Se genera un nuevo chunk a partir de la siguiente oracion.

3.2 Justificacion Teorica (Ref: Notas sobre PLN, Cap. 14)

Segun el libro de referencia Notas sobre Procesamiento de Lenguaje Natural (Manrique, 2026), la estrategia de fragmentacion tiene impacto directo en la calidad de la recuperacion (Seccion 14.8). El texto indica que los fragmentos deben contener una idea o argumento cohesivo para que el vector capture con precision su significado. Nuestra estrategia por oraciones cumple este principio al garantizar unidades linguisticamente completas.

3.3 Metadata por Fragmento

Cada chunk almacena los 8 campos obligatorios de la Tabla 2 de la especificacion:

- doc_id: Identificador unico del documento de origen.
- chunk_id: Identificador unico del fragmento (formato DOC_ID-chunk-NNNN).
- fuente: Ruta relativa del archivo original provisto por ADL.
- formato: Tipo de archivo (pdf, html, json, csv, xlsx, txt, md).
- fenomeno: Fenomeno tematico al que pertenece (1, 2 o 3).
- posicion: Indice ordinal del fragmento dentro del documento (base 0).
- num_tokens: Numero de tokens del fragmento.
- texto: Texto original del fragmento sin modificaciones.

4. Codificación Semántica: Encoder Seleccionado

Se seleccionó el modelo intl-multilingual-e5-small-v2 (disponible públicamente en HuggingFace con licencia MIT) como encoder principal. Los criterios de selección fueron:

- Soporte multilingüe nativo: El modelo está entrenado en más de 100 idiomas, incluyendo español, inglés y portugués. Produce representaciones comparables entre idiomas, permitiendo que una consulta en español recupere documentos en inglés o portugués.
- Dimensionalidad: 384 dimensiones. Este tamaño ofrece un balance óptimo entre expresividad semántica y eficiencia computacional.
- Longitud máxima de entrada: 512 tokens, compatible con nuestra estrategia de chunking.
- Rendimiento en benchmarks: Ranking competitivo en MTEB y BEIR en tareas de recuperación multilingüe, superando a modelos de mayor tamaño en relación calidad/costo.
- Eficiencia computacional: Con solo 33M de parámetros, permite indexar el corpus completo de 187,907 fragmentos en aproximadamente 16 minutos utilizando GPU (NVIDIA RTX 2080 SUPER).
- Licencia: MIT, totalmente compatible con el contexto del reto.

4.1 Normalización de Vectores

Los vectores generados por el encoder se normalizan a norma unitaria (L2) antes de insertarlos en el índice FAISS. Esto permite que la similitud coseno sea equivalente al producto interno (Ecuación 4 de la especificación), maximizando la eficiencia de IndexFlatIP.

5. Gestión del Índice Vectorial con FAISS

Se utiliza el tipo de índice IndexFlatIP (Inner Product) de FAISS. Esta elección se justifica por:

- Búsqueda exacta: Garantiza que los resultados sean los k vecinos más cercanos reales, sin pérdida de precisión por aproximación.
- Volumen del corpus: Con 187,907 vectores de 384 dimensiones, el tamaño del índice es manejable (~275 MB) y las búsquedas se completan en milisegundos.
- Equivalencia con similitud coseno: Al utilizar vectores normalizados, IndexFlatIP maximiza el producto interno, equivalente a maximizar la similitud coseno (Sección 8.2 de la especificación).

5.1 Persistencia y Reproducibilidad

El índice se serializa con `faiss.write_index()` en el archivo `index.faiss`. La metadata se persiste en un archivo `metadata.jsonl` en formato JSON Lines, con exactamente una línea por fragmento. El orden de las líneas corresponde al identificador interno asignado por FAISS durante la indexación.

6. Módulo de Recuperación Híbrida

El sistema implementa un pipeline de recuperación híbrida que combina búsqueda densa (vectorial) con búsqueda dispersa (lexica), fusionadas mediante Reciprocal Rank Fusion (RRF). Esta arquitectura está fundamentada en la Sección 14.9 del libro de referencia y en la Sección 8.4 de la especificación técnica.

6.1 Búsqueda Densa (FAISS)

La consulta del usuario se codifica con el mismo encoder (intl-e5-small-v2) utilizado durante la indexación. El vector resultante se normaliza y se buscan los 50 fragmentos más similares mediante IndexFlatIP.

6.2 Búsqueda Lexica (BM25)

Se implementa un recuperador BM25 (`lib/bm25_retriever.py`) con soporte multilingüe. Las stopwords se cargan para español, inglés y portugués. El tokenizador divide el texto en tokens alfanuméricos en minúsculas. Para cada consulta, se recuperan los 50 fragmentos con mayor puntuación BM25.

6.3 Fusion: Reciprocal Rank Fusion (RRF)

Las dos listas de candidatos (densa y lexica) se fusionan mediante RRF con constante de suavizado $k_0=60$ (Ecuación 7 de la especificación). RRF fue elegido sobre CombSUM y CombMNZ porque es robusto a las diferencias de escala entre las puntuaciones de similitud coseno (rango $[-1,1]$) y las puntuaciones BM25 (rango variable). Como indica el libro de referencia (Sección 14.9), la señal semántica densa corrige sinonimia y parafrasis, mientras que la señal dispersa preserva términos críticos, entidades y coincidencias exactas.

6.4 Agregación a Nivel de Documento

Para construir los 3 documentos más relevantes, se utiliza max pooling: la puntuación de un documento es la máxima puntuación RRF entre todos sus fragmentos recuperados. Los documentos se ordenan de mayor a menor y se seleccionan los 3 primeros.

6.5 Límite de 250 Palabras

Cada fragmento en la salida se verifica contra el límite de 250 palabras. Si un chunk excede este límite, se trunca retrocediendo al final de la última oración completa dentro del límite, cumpliendo el requisito de completitud lingüística.

7. Mejoras Propuestas para la Etapa 2

Basandonos en el analisis del libro de referencia Notas sobre Procesamiento de Lenguaje Natural (Manrique, 2026), se identifican las siguientes optimizaciones para la Etapa 2:

7.1 Chunking con Ventana Deslizante (Ref: Seccion 14.8)

Se propone reemplazar la segmentacion actual por oraciones con una estrategia de ventana deslizante (sliding window) con solapamiento del 20%. Esto permite que el contexto compartido entre chunks consecutivos reduzca la perdida de informacion en los bordes de fragmentacion, mejorando la recuperacion de ideas que abarcan multiples oraciones.

7.2 Multi-Query RAG (Ref: Seccion 14.9)

Se propone integrar un modulo de reformulacion de consultas que genere 3 variantes de cada pregunta (sinonimos, parafrasis, expansion de terminos) para ampliar la cobertura documental. Las tres busquedas se unifican mediante RRF. El libro indica que esta tecnica reduce la sensibilidad a una sola formulacion y mejora significativamente el recall.

7.3 Grafo de Conocimiento (Ref: Seccion 7 del reto)

Como componente bonus, se propone construir un grafo de conocimiento utilizando NER multilingue (HuggingFace) para extraer entidades (países, organizaciones, tecnologías) y relaciones entre ellas. El grafo se integraria como una fuente adicional de candidatos en el pipeline de recuperacion, fusionandose via RRF como un indice adicional (Seccion 8.5 de la especificacion).

7.4 Evaluacion de Groundedness (Ref: Seccion 14.10)

Para la Etapa 2 conversacional, se propone implementar el framework LLM-as-a-judge para evaluar sistematicamente la fidelidad (faithfulness) y el anclaje (groundedness) de las respuestas generadas respecto al contexto recuperado, mitigando alucinaciones.

8. Estadisticas del Corpus Procesado

- Total de documentos procesados: 1,695 (tras deduplicacion)
- Total de fragmentos generados: 187,907
- Tamano del indice FAISS: 275 MB (IndexFlatIP, 384 dims)
- Tamano del almacen de metadata: 266 MB (JSON Lines)
- Encoder: intl-multilingual-e5-small-v2 (33M parametros, 384 dims)
- Tiempo de indexacion: ~16 minutos con GPU NVIDIA RTX 2080 SUPER
- Estrategia de fusion: Reciprocal Rank Fusion (k0=60)
- Idiomas soportados: Espanol, Ingles, Portugues
- Restriccion cumplida: Ningun modelo generativo (decoder) utilizado en ninguna etapa

9. Conclusion

El sistema construido implementa un pipeline de recuperacion hibrida robusto que combina la precision semantica de los embeddings densos con la especificidad lexica de BM25. La eleccion de intl-e5-small-v2 como encoder garantiza soporte multilingue nativo para los tres idiomas del corpus. La fusion mediante RRF asegura robustez ante diferencias de escala entre metricas. El cumplimiento estricto de todos los requisitos obligatorios (completitud linguistica, metadata, formato de salida, ausencia de modelos generativos) posiciona al equipo para obtener resultados competitivos en

ambas metricas de evaluacion.

10. Referencias

- Manrique, R. (2026). Notas sobre Procesamiento de Lenguaje Natural. Primera Edicion. Capítulos 12 y 14.
- CODEFEST AD ASTRA 2026. Especificacion Tecnica Etapa 1, Version 1.0. Universidad de los Andes / Fuerza Aeroespacial Colombiana.
- Wang, L. et al. (2022). Text Embeddings by Weakly-Supervised Contrastive Pre-training (E5). arXiv:2212.03533.
- Johnson, J., Douze, M., Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE TBD.
- Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. FnTIR.
- Cormack, G., Clarke, C., Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. SIGIR 2009.